

DEVLORE: Device Interrupt Protection for Confidential VMs

Andrin Bertschi* Supraja Sridhara* Mark Kuhne Benedict Schlüter
 Friederike Groschupp Clément Thorens Nicolas Dutly Srdjan Capkun Shweta Shinde
ETH Zurich

Abstract

Modern confidential computing executes sensitive computation in an abstraction called confidential VMs and protects from the hypervisor, host OS, and other co-resident VMs. It has been shown that an attacker can inject malicious interrupts to break the confidentiality and integrity of confidential VMs. We present DEVLORE, a device interrupt isolation mechanism that protects confidential VMs from interrupt manipulation attacks. Our design employs a delegate-but-check strategy by offloading interrupt management to the hypervisor, but adds correctness checks in the trusted software. We prototype our design on Arm Confidential Computing Architecture (CCA). We evaluate it on Arm FVP to demonstrate four diverse devices attached to confidential VMs and report costs on a Rock5b board. Our case studies show the feasibility of real-world use cases and that DEVLORE incurs minimal overheads of 0.06% for typical integrated GPU applications.

1 Introduction

Arm-based platforms are deployed in consumer electronic devices (e.g., phones, cars, laptops), edge devices, and the cloud. Applications running on these platforms typically use integrated devices (e.g., display, sensors, integrated GPUs) to perform sensitive tasks (e.g., inference, authentication) on-device or at edge. Cloud providers offer integrated accelerators connected to Arm CPUs (e.g., Grace-Hopper and Grace-Blackwell integrate Arm processors with state-of-the-art GPUs [49, 51]). In these settings, it is important to safeguard users’ sensitive computation on the platform from untrusted software (e.g., hypervisor executing on personal devices) and untrusted providers (e.g., cloud management).

Confidential computing enables users to execute their computation in isolation from privileged untrusted software. Arm Confidential Computing Architecture (CCA), similar to Intel TDX and AMD SEV-SNP, provides a confidential virtual machine (CVM) abstraction with a new hardware extension

called realm management extensions (RME) [5, 13, 37]. CCA isolates CVMs from each other as well as from untrusted hypervisors and host OSes. To achieve this, CCA introduces two new worlds called the realm and the root world. The root world has the highest privilege and exclusively executes the trusted firmware including the monitor. The realm world houses the CVMs and a Realm Management Monitor (RMM). These new worlds are in addition to two existing worlds: the normal/non-secure world that is reserved for the host hypervisor and the secure world for trusted OS and apps.

Arm CCA-based CVMs executing in the realm world may need access to integrated devices on both consumer devices and the cloud. For example, to use a keypad or biometric sensor to perform authentication or to use an integrated GPU for computation on sensitive data in the cloud. However, CCA prohibits devices from accessing CVM memory. Several recent works address this gap by allowing devices and CVMs to directly access each others’ memory. They enforce memory isolation guarantees to ensure that device access does not compromise the CVM confidentiality or integrity [57, 61, 67]. On the other hand, recent works have also shown that interrupt injection attacks from a malicious hypervisor can compromise the CVM execution [58, 59].

To address this gap, we aim to enforce interrupt isolation—malicious interrupts injected by the attacker are never delivered to the CVMs, while ensuring that benign device interrupts are delivered correctly. Interrupt isolation presents challenges that are different from device memory isolation. For memory isolation, each device belongs to exactly one CVM. So, once the device is attached via memory setup, the hardware enforces the isolation without further software intervention. In contrast, the interrupt controller and its state is one resource that needs to be shared and multiplexed between the hypervisor and all the CVMs. Further, we not only need to isolate the interrupt configuration but also runtime delivery and acknowledgment.

We consider several potential designs to achieve this seemingly simple primitive of interrupt isolation. Our design exploration shows that obvious solutions are insufficient. Fur-

* These authors contributed equally to this work.

ther, since enforcing the primitive entails reasoning about the entire interrupt lifecycle (registration, physical to virtual conversion, delivery, priority, acknowledgment), we have to weigh the design options for each phase. We isolate both physical and virtual interrupts without any software changes to existing device drivers. Our insight is to delegate the interrupt management and delivery to the untrusted hypervisor while the trusted software checks if the hypervisor misbehaves. Our approach minimizes software changes because of this delegate-but-check strategy while maintaining low performance overheads.

We prototype our design, called DEVLORE, on the Fixed Virtual Platform (FVP), a CCA-enabled emulator provided by Arm to demonstrate feasibility, correctness, and compatibility with hardware specification and software stack [10]. Since CCA is not yet available in production CPUs that are accessible to researchers [15, 16, 18], we retrofit DEVLORE changes to a RK3588 Rock5b board with Armv8.2-A to estimate DEVLORE performance. DEVLORE changes to the hypervisor, firmware, and the kernel (host and guest) are minimal ($\approx 7k$ LoC), and without device driver changes.

We test the functional correctness of DEVLORE using our prototype on the FVP with four devices of varying complexity and isolation requirements: GPU, UART, LED, and Keyboard. Further, our three case-studies on the FVP show that DEVLORE has non-invasive impact on applications (IRC, terminal browser, text editor). Our measurements on the Rock5b board performance prototype show that DEVLORE overheads are minimal. First, we run interrupt stress workloads using GPU and UART on the Rock5b board. Our experiments show that under sustained interrupt load, DEVLORE only incurs overheads up to 1%. Next, our case study using the glmark2 GPU benchmark that models typical integrated GPU applications shows that DEVLORE overhead drops to only 0.06%.

Contributions. The main contributions of this paper are:

Interrupt Isolation. We propose an interrupt isolation primitive to bridge the gap between memory and interrupts for integrated devices attached to Arm CCA CVM.

Delegate-but-check. Our design, DEVLORE, delegates the interrupt management to the untrusted hypervisor while the trusted software checks its correctness.

Devices. DEVLORE isolates device interrupts with no modifications to applications or device drivers and limited overheads. We demonstrate DEVLORE with devices and case studies evaluated on both the Arm FVP and an Arm board.

2 Background

We explain memory and interrupt behavior of integrated devices on Arm and give an overview of CCA.

Arm CCA. To enable Arm CCA, the Arm ISA adds support for Realm Management Extensions (RME). RME enables

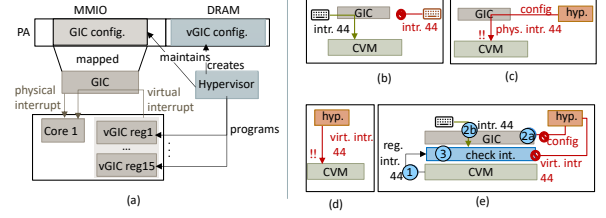


Figure 1: (a) Arm interrupt architecture where the hypervisor maintains the GIC configuration, creates and maintains vGIC configuration for CVMs, and programs vGIC system registers. (b) Devices cannot forge interrupt numbers that are not assigned to them. (c) Hypervisor configures the GIC to inject fake physical interrupt. (d) Hypervisor injects fake virtual interrupt into the CVM. (e) DEVLORE protections to 1. register interrupts for isolation, 2a-2b. record physical interrupts after protecting the GIC, and 3. check virtual interrupts.

computation to execute in one of the four worlds: normal, realm, root, or secure. It extends the previous two worlds (normal and secure) with two new worlds (realm and root) and isolates them. To enforce world-level isolation, RME introduces hardware units called Granule Protection Checks (GPCs) where a granule is the smallest addressable chunk of memory. The GPCs filter all memory accesses by looking up a mapping from the granule to its physical address space (PAS) as stated in the Granule Protection Table (GPT). The GPT is stored in the root world and programmed by the monitor. Apart from the worlds, the Arm architecture enables privilege levels called exception levels (EL) from EL0 to EL3 as shown in Fig 2.a. In the realm world, CCA enables running CVMs (in EL0 and EL1) that are not accessible to the normal world and deploys a Realm Management Monitor (RMM) in realm EL2. The RMM isolates the CVMs using stage-two translation tables (S2 tables). It exposes a Realm Service Interface (RSI) and Realm Management Interface (RMI) for the CVMs and the normal world hypervisor respectively (Fig 2.a).

DMA-capable devices with Arm CCA. On the Arm platform, DMA-capable devices are behind an SMMU (equivalent to an IOMMU on Intel/AMD). CCA blocks all devices from directly accessing realm memory. To do this, RME adds GPCs to the SMMU and deems all memory accesses from devices as originating in the normal world, thus effectively blocking all device access to realm memory. To attach devices to CVMs, prior works overcome this limitation by using encrypted bounce buffers or by leveraging the GPCs to create different views of memory. To use encrypted bounce buffers, prior works first encrypt the data to send to the device and copy it to normal world device accessible memory and vice versa [50]. Works that leverage the GPCs to create different views of memory, present the device memory shared with the CVM as realm world memory to the cores and as normal

world memory to the devices (Sec A.3) [57, 61, 67].

Arm Interrupt Management. The Generic Interrupt Controller (GIC) delivers physical interrupts to the cores based on its configuration (e.g., enabled/disabled interrupts, order based on interrupt priorities). With virtualization support, the hypervisor manages the GIC, which exposes a memory-mapped configuration address range. The hypervisor can use the GIC configuration space to selectively enable/disable interrupts, set up interrupt affinities, and program interrupt priorities. With virtualization, VMs running on the cores can only handle virtual interrupts. So, the hypervisor virtualizes the physical interrupts from the GIC by programming virtual GICs (vGICs) provided by the platform. The vGICs create local views of the GIC for each VM (see Fig 1.a). To send a virtual interrupt to a VM, the hypervisor programs up to n vGIC system registers, where n depends on the GIC implementation. If there are several pending interrupts, the hypervisor programs the vGIC with the highest priority interrupts. On VM entry, the vGIC fires the virtual interrupts programmed by the hypervisor in the vGIC system registers.

Arm architecture defines three classes of event notifications: synchronous exceptions, interrupts (IRQ, FIQ), and SEErrors. Heckler concluded that synchronous exceptions and SEErrors are traps that the trusted hardware generates and cannot be injected externally (either from hardware or software) [59]. This leaves us with the interrupts that the hypervisor can fake. We can split these interrupts further into three categories: timer interrupts, Inter-Processor-Interrupts (IPIs), and device interrupts. Heckler also concluded that the hypervisor cannot use timer interrupts and IPIs to break CVM security, even on TDX and SEV-SNP. Therefore, in this work we focus on ensuring device interrupt isolation.

CCA Interrupt Support. CCA expects the hypervisor to virtualize and manage interrupts for the CVMs. However, directly switching to the hypervisor when a CVM core receives a physical interrupt can leak CVM state. So CCA ensures that all physical interrupts to cores executing CVMs trap to the RMM. The RMM saves the state of the CVM, and then switches to the hypervisor to virtualize the interrupt. For security, CCA does not allow the hypervisor to directly program the vGIC system registers and the RMM clears any values that the hypervisor might have written to these registers before entering a CVM. Instead, to program the vGIC for the CVMs, the hypervisor invokes the RMM with the vGIC configuration. On each RMM invocation, the hypervisor can program the vGIC to send at most n distinct pending interrupts. In response, the RMM checks if the vGIC configuration is valid according to Arm GIC specification and programs the CVM vGIC system registers. Therefore, currently the RMM allows the hypervisor to inject all interrupts to the CVM.

3 Problem & Insights

We motivate and scope the need for interrupt isolation.

3.1 Injecting Malicious Device Interrupts

We discuss how an attacker can inject fake interrupts to mount interrupt attacks.

Recent Interrupt-Based Attacks. Recent works have shown that a malicious hypervisor can use interrupt injections to compromise CVM security [58, 59]. They observe that confidential computing solutions (e.g., Intel TDX, AMD SEV-SNP, Arm CCA) delegate CVM management (e.g., scheduling, memory management, interrupt management) to the untrusted hypervisor. To manage interrupts and virtualize them for CVMs, the hypervisor controls the interrupt controller (called GIC on Arm platforms) that delivers interrupts to the cores. So, the hypervisor can program the GIC to inject interrupts directly into the CVMs at any point during their execution. Since the CVM is not aware of this possibility, similar to Iago attacks [23], it breaks execution integrity (Sec 2). Next, we discuss these potential threats for CVMs that connected to devices.

Untrusted Devices. On any given Arm platform, an integrated device can only assert its designated interrupt line. So, integrated devices cannot send physical interrupts to fake interrupts from other devices that might be attached to CVMs (Fig 1.b). Therefore, even if an attacker controls an integrated device they cannot use it to inject fake interrupts. Next, we discuss threats from untrusted management software (e.g., hypervisors) that recent works leverage to mount interrupt-based attacks [58, 59].

Untrusted Management Software. The hypervisor manages the interrupts for the CVMs and can abuse it in two ways: (i) configure the GIC to trigger fake physical interrupts (Fig 1.c); and (ii) use virtual interrupt injection through the vGIC system registers to deliver fake virtual interrupts of either type (Fig 1.d). Next, we analyze device drivers to understand how they handle interrupts.

3.2 Driver Examples

To understand how device drivers handle interrupts, we manually analyzed the open-source drivers of popular integrated devices in the Linux kernel.

Counter. The Linux kernel counter subsystem implements an interrupt-driven counter. This counter can use any interrupt source (e.g., temperature sensor, proximity sensor) as an event source and increments a counter when it receives an interrupt (Lst 1). The counter implementation can be used by user-space applications to be notified when the counter is incremented, e.g., when an event has occurred for a fixed number of times (Line 5). Assume that the interrupt source for this counter is connected to a CVM and this counter is used by a CVM application. The untrusted hypervisor can maliciously inject interrupts, trigger this interrupt handler, and increment the counter to trigger the CVM application event condition eventually. Thus, using malicious interrupt injection, the hypervisor successfully tricks the CVM into believing an event

occurred when, in reality, it did not, breaking its execution integrity.

```

1 irqreturn_t interrupt_cnt_isr
2     (int irq, void *dev_id){
3     struct counter_device *ctx = dev_id;
4     struct interrupt_cnt_priv *priv = counter_priv(ctr);
5     atomic_inc(&priv->count);
6     counter_push_event(ctr,
7         COUNTER_EVENT_CHANGE_OF_STATE, 0);
8     return IRQ_HANDLED; }

```

Listing 1: Linux interrupt-driver counter.

Wireless Device Boot-up. Qualcomm WCNSS chip is used in a wide range of devices with Arm cores for wireless communications and is vulnerable to the untrusted hypervisor interrupt injection attacks. Concretely, the WCNSS driver used for verified firmware loading expects interrupts to indicate when the device firmware is successfully loaded and ready to use (see Lst 2) [47].

```

1 irqreturn_t wcnss_ready_interrupt
2     (int irq, void *dev){
3     struct qcom_wcnss *wcnss = dev;
4     complete(&wcnss->start_done);
5     return IRQ_HANDLED; }

```

Listing 2: Ready interrupt handler in WCNSS driver.

If the hypervisor injects an interrupt to trigger this handler before the device is ready, it tricks the driver into transitioning the device state to ready. This may have undesirable effects (e.g., CVM may use an outdated or corrupted WiFi firmware image). Thus, the hypervisor may break the driver execution integrity using only malicious interrupt injection.

3.3 Threat Model and Scope

We trust the platform and all of its components, including the GIC. Like previous works [57, 67], we assume that the hardware integration of the devices on the platform conforms to the specification and is bug-free, i.e., devices are only accessible through the specified address ranges and can only assert their assigned interrupts. We assume that all software executing in the normal and secure worlds is untrusted. We trust the CCA hardware, assume that all trusted CCA software (i.e., the RMM and monitor) is implemented according to CCA specifications, and assume mutual distrust between CVMs. For a given CVM, we consider all software executing in CVM (i.e., operating system, device drivers and runtimes, applications) as trusted. Protecting against side-channels and microarchitectural attacks is orthogonal to our work and we consider them to be out of scope.

Assumptions. DEVLORE uses techniques from existing works to attach devices to a CVM, ensure runtime memory isolation, and detach devices from the CVMs [12, 57, 61]. For security, these prior works provide the below guarantees.

G_{id}: Identity. Attackers cannot forge device identities and physical memory used to access a device is fixed by the platform integration.

G_a: Device Attach. Devices are attached to a CVM in a trusted state (including device reset and attestation).

G_{mem}: Memory Isolation. Only the device and the CVM it is attached to can access the device memory.

G_d: Device Detach. Detaching devices from CVM will not compromise CVM security (e.g., leak residual device state).

Problem Statement. We aim to provide guarantees for interrupts from integrated devices delivered to a CVM. Concretely, we ensure that the device interrupts delivered to a CVM in the presence of an untrusted hypervisor is equivalent to interrupts that would be delivered if the hypervisor was trusted. This guarantees that fake interrupts injected by a malicious hypervisor cannot compromise CVM security.

4 Potential Solution

Our analysis of the device drivers in the Linux kernel shows that in addition to pure interrupts, the devices use two other types of event notification mechanisms: (1) only poll memory-mapped regions, or (2) use interrupts in conjunction with memory-mapped region reads. In both these cases, the interrupt handler reads or checks isolated MMIO memory before processing the interrupt. This isolated MMIO memory is not accessible to the malicious hypervisor. Therefore, if the drivers check some state in the device MMIO memory before processing the interrupts, it can prevent the interrupt attacks from Sec 3.

If all device drivers adopt one of the two mechanisms outlined above and the MMIO regions are isolated, they can thwart malicious interrupt injection. However, this solution is infeasible in some cases, such as the interrupt-driven counter, which purely depends on an interrupt source and is device agnostic. Because of the device-agnostic nature of this counter, reading from an MMIO space is not possible, as different devices have different MMIO spaces and behaviors. Therefore, supporting this interrupt-driven counter in a CVM such that the hypervisor cannot corrupt it requires architectural guarantees for interrupt isolation. For devices where such memory-mapped reads would be possible (e.g., Qualcomm WCNSS chip), doing so may incur hardware changes to the device to write to the register.

To adopt this solution, we will need to change any device that simply raises an interrupt when an event occurs to also write to a memory-mapped region. Further, we will need to change all existing device drivers where such a check is feasible. Doing this will incur significant changes to the hardware and manual driver patching. More importantly, *even one device left unchanged or one driver left unpatched* will leave the CVM vulnerable to interrupt attacks. We observe that such invasive hardware and driver changes are not necessary if we can provide architectural guarantees for interrupt delivery.

Therefore, in DEVLORE we take a principled approach to ensure isolated interrupt delivery to the CVMs.

5 Solution Overview

First, we provide an intuition for our solution. On the Arm platforms, untrusted integrated devices cannot fake physical interrupts from other devices as they can only assert their designated interrupt lines (Fig 1.b). The hypervisor can inject physical interrupts only by writing to the GIC directly (Fig 1.c). So, by constraining the hypervisor capability to maliciously write to the GIC, we guarantee that all registered physical device interrupts originate solely from their respective devices. Using this observation, in DEVLORE we first prevent malicious GIC writes from the hypervisor to ensure physical device interrupt authenticity (Sec 5). However, physical interrupt authenticity alone is not sufficient. We also need to isolate the virtual interrupts that are delivered to the CVM. In CCA, the hypervisor performs interrupt management. So, the hypervisor can always inject fake virtual interrupts to the CVMs. To prevent malicious virtual interrupt injection, we only allow registered virtual interrupt injection if there was a corresponding authentic physical interrupt from the device (Fig 1.e).

Registered Interrupts for Isolation. CVMs must *register* specific device interrupts to isolate from the hypervisor using DEVLORE. We call these interrupts the CVM *registered interrupts*. To isolate these interrupts, DEVLORE records all registered physical device interrupts to create a ground truth (Sec 5.1). Then, when the hypervisor injects virtual interrupts, DEVLORE uses the ground truth to ensure that all registered virtual interrupts that reach the CVM have a corresponding authentic physical interrupts (Sec 5.2).

Prevent Malicious GIC Writes. Our first step is to prevent a malicious hypervisor from injecting physical interrupts by writing directly to the GIC. DEVLORE stops the hypervisor from directly writing to the GIC memory by marking the GIC memory as root world and using the monitor to check any updates to the GIC configuration. This ensures the physical interrupt authenticity, i.e., any physical device interrupt that arrives at the cores is guaranteed to only be from the device.

5.1 Recording Physical Interrupts

Once we have established the authenticity of physical device interrupts, we need a mechanism to record these interrupts when they arrive at the cores. By default, the GIC sends a physical interrupt to any core that is active. In CCA, all physical interrupts are forwarded to the hypervisor for interrupt management. To record registered physical interrupts, we need to ensure that these physical interrupts are always delivered to a trusted software (e.g., RMM, CVM, monitor). Then, they can be recorded by the trusted software, before being forwarded to the hypervisor.

Option 1: Deliver to CVM. We can consider delivering the physical interrupt directly to the CVM that it belongs to. However, this is not feasible as the CVM runs in a virtualized environment and cannot directly handle physical interrupts. Instead, on receiving the physical interrupt the hardware forces the CVM to exit and traps the interrupt to the RMM (in EL2). Therefore, it is impossible for the CVM to record these physical interrupts.

Option 2: Deliver to RMM. Physical interrupts always trap to EL2. So, another potential solution is to ensure that the registered physical interrupts are always delivered to the RMM which executes in realm EL2. To guarantee isolated interrupt delivery to RMM, we have to ensure that the interrupt is only delivered to cores that are executing in the realm world. However, ensuring this is not straightforward. In CCA, the hypervisor is responsible for all scheduling decisions. So, at any point in time, any given core can be executing in any world. To ensure reliable physical interrupt delivery to the RMM, we need to get around this dynamic scheduling behavior. For this, we can pin a core that always executes the RMM and program the GIC to only deliver the registered device interrupts to this core [22]. With this the RMM can record all physical interrupts that arrive, and then forward the interrupt to the hypervisor. While this approach allows us to record the interrupts, it wastes a core (because of pinning) and makes the pinned core a bottleneck.

Option 3: Change the GIC Hardware. In Option 2, we needed to pin a core with the RMM because the GIC cannot be configured to deliver specific interrupts only to realm cores. We can overcome the need for core-pinning by changing the GIC hardware to deliver specific interrupts to realm cores only. We observe that the GIC has such a feature for the secure world. Using this feature, the secure world software can configure the GIC to deliver specific interrupts only to the secure world. With hardware changes to the GIC, we can introduce an analogous feature for the realm world. Then, the RMM configures the GIC to deliver all registered interrupts to realm cores only. Now, registered interrupts are always delivered to the RMM and we can use the RMM to record all physical interrupts. However, the GIC is a highly optimized piece of hardware and to propose changes to it, we need access to the hardware and tools which we do not have. More importantly, these hardware changes will not be sufficient to ensure virtual interrupt isolation (see Sec 5.2). Therefore, in DEVLORE we devise a mechanism to record the registered physical interrupts without hardware changes.

DEVLORE Solution: Deliver to Monitor. By default, the GIC provides a mechanism to ensure that specific interrupts are only delivered to EL3 i.e., the monitor. We use this feature and configure the GIC using the monitor such that all registered interrupts are delivered to the monitor (Step 1 in Fig 2.b). Then, when the registered physical interrupt arrives, the monitor records the physical interrupt in a realm data structure (Step 2). After recording, the monitor forwards the

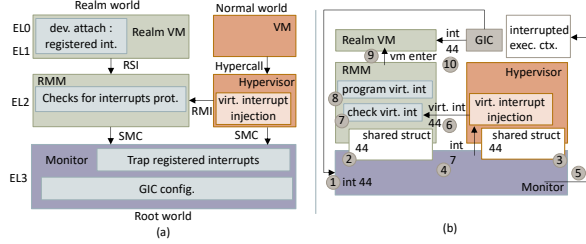


Figure 2: DEVLORE Design. (a) Interrupt Isolation for devices attached to CVMs (b) DEVLORE interrupt delivery flow for interrupt 44. Steps (1-5): record physical interrupts. Step (6) Hypervisor virtualizes interrupt. Steps (7-9): RMM checks and delivers virtual interrupts. 1. Interrupt 44 traps to monitor. 2,3. Monitor writes 44 to RMM and Hypervisor data structures, 4. notifies the hypervisor using interrupt 7, 5. and returns to the interrupted app. 6. Hypervisor sends virtual interrupt configuration to the RMM. 7. RMM checks the virtual interrupt configuration, 8. programs the vGIC system registers, 9. and enters the CVM. 10. On CVM entry, the GIC fires interrupt 44.

interrupt to the hypervisor for further processing (Step 3–4). Finally, it returns to the interrupt execution context (Step 5). The realm data structure created by the monitor serves as our ground truth to check virtual interrupts.

5.2 Checking Virtual Interrupts

Once we have established the authenticity of recorded physical interrupts, we need to isolate the virtual interrupts.

Option 1: Move Interrupt Management to the RMM. CCA employs a delegate-but-check strategy for all CVM management. So, it lets the hypervisor perform interrupt management for the CVMs. We can move the virtual interrupt management to the RMM such that the hypervisor has no opportunity to perform malicious interrupt delivery. Now, the RMM is responsible for all virtual interrupt management, including the non-registered interrupts (e.g., timers, IPIs). Moving the management of all interrupts to the RMM is a non-trivial task as these interrupts are tightly coupled with the hypervisor CVM scheduling and would require invasive changes.

Instead, we can move only the registered interrupt management to the RMM. To do this, the monitor records the physical interrupts (Sec 5.1) and then forwards all registered interrupts to the RMM. Then, when the CVM is scheduled, the RMM can program the CVM vGIC system registers (Sec 2) to inject the virtual interrupts. We observe that the same vGIC system registers are also used by the hypervisor to forward other non-registered virtual interrupts (e.g., timers, IPIs) to the CVM. Because the number of vGIC system registers are limited, this approach leads to contention between the RMM

and the hypervisor for the same registers. More importantly, the RMM now has to reason about priority of interrupts for all the interrupts (registered and non-registered) that are to be sent to the CVM. This further complicates the design and require the RMM to hold some interrupts to be sent to the CVM in a queue introducing queue management complexity. We conclude that this approach, while feasible, is not practical and breaks CCA delegate-but-check strategy.

DEVLORE Solution: Check in the RMM. By default, CCA does not allow the hypervisor to directly program the CVM vGIC system registers. Instead, every time the hypervisor schedules a CVM, it can request the RMM to program the CVM vGIC to inject the virtual interrupts (Step 6). In DEVLORE, the RMM rigorously checks that the virtual interrupt injection is benign. To perform this check, the RMM looks up the data structure with authentic physical interrupts from the monitor (Step 7 in Fig 2.b). This way, the RMM can distinguish between the case where the hypervisor is injecting a virtual interrupt corresponding to an authentic physical interrupt versus a malicious virtual interrupt. Thus, DEVLORE ensures that the hypervisor cannot inject arbitrary virtual interrupts while preserving ordering and interrupt priorities as we explain in detail in Sec 6.2. DEVLORE design ensures that even if the interrupt management remains in the hypervisor, it cannot break CVM security (Sec 7).

6 Design

We detail DEVLORE interrupt isolation mechanism.

Device Lifecycle. DEVLORE extends the device attach process of prior works to allow CVMs to enable DEVLORE interrupt isolation [12, 57, 61]. Generally, a single device may generate multiple physical interrupt for different functions. For example, a Mali GPU can raise a total of three interrupts, one each for power management, on page-faults, and job completion. By default, the hypervisor provides the virtual interrupt information to CVMs in the form of a mapping from $(devID, function)$ to virtual interrupt number. With DEVLORE, the platform manufacturer provides a trusted mapping from $(devID, function)$ to platform-specific physical interrupt number. We observe that the $(devID, function)$ tuple is unique on any given platform. So, the RMM can use this tuple to securely manage interrupt registration. Concretely, we have to ensure that although the physical interrupt numbers vary across platforms and the hypervisor may choose different virtual interrupt numbers for CVMs, the hypervisor cannot abuse this to tamper with the virtual-to-physical interrupt bindings.

When a CVM wants to register an interrupt, it requests the RMM with $(devID, function)$ and its virtual interrupt number. On receiving this request, the RMM first checks that the device is attached to the requesting CVM before proceeding. If the check passes, the RMM now needs the physical interrupt that corresponds to the $(devID, function)$ in the CVM

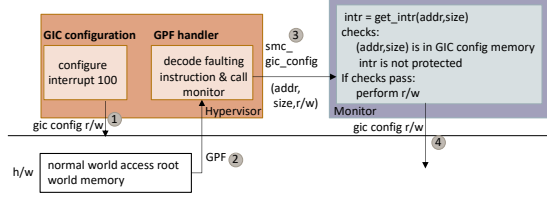


Figure 3: Hypervisor GIC configuration flow with DEVLORE. 1. Hypervisor tries to configure GIC 2. This operation tries to access root memory so the hardware raises a Granule Protection Fault (GPF) 3. that the hypervisor traps on. In the trap handler, the hypervisor reconstructs the faulting instruction and invokes the monitor to perform the GIC config. operation 4. The monitor checks the GIC config.

request. For this, the RMM looks up the trusted platform-specific mapping from $(devID, function)$ to physical interrupt numbers. This look up ties the CVM-specific virtual interrupt number to the platform-specific physical interrupt number. The RMM uses this physical interrupt number to invoke the monitor to register the interrupt for DEVLORE isolation. Once registered, the RMM stores the virtual interrupt number to facilitate the RMM checks on the virtual interrupt as explained in Sec 6.2. Finally, the RMM adds the registration information (i.e., $(devID, function)$, physical interrupt number, virtual interrupt number) to the CCA attestation report which a remote verifier can check.

On device detach, the RMM checks if interrupt isolation was enabled for the device. If so, the RMM invokes the monitor to update the GIC configuration to de-register the interrupt isolation.

6.1 Ensuring Interrupt Isolation with DEVLORE

For registered device interrupts, DEVLORE guarantees interrupt isolation (configuration, delivery, acknowledgment) under an untrusted hypervisor that performs interrupt management. DEVLORE proceeds in 3 steps (Sec 5): establishing authenticated physical interrupts, recording the authenticated physical interrupts to as ground truth, and checking virtual interrupts against the ground truth.

Authenticated Physical Interrupts. In Arm CCA, the hypervisor manages the GIC by default and can maliciously modify its configuration to inject interrupts. To circumvent this threat, DEVLORE moves the GIC configuration in the root world and protects it using the monitor checks. To maintain hypervisor functionality, DEVLORE introduces a GIC configuration interface for the hypervisor to use in the monitor (see Fig 3). Concretely, on each GIC configuration request from the hypervisor, the monitor checks if the hypervisor is not trying to reconfigure the GIC for a registered interrupt. If the

check passes, the monitor allows the configuration request through. With this, DEVLORE can ensure that all registered physical interrupts that the GIC injects are authentic i.e., from their respective devices. But this alone will not provide interrupt isolation guarantees to the CVM. DEVLORE has to reason about both physical and virtual interrupt security for the registered device interrupts.

Recording Registered Interrupts. Once DEVLORE establishes physical interrupt authenticity, it uses this as the ground truth to check that the hypervisor benignly virtualizes them for the CVM (Sec 5). To create the ground truth, DEVLORE has to record all the registered physical interrupts that the GIC fires. For this, it configures the GIC such that all registered device interrupts trap to the monitor. This allows the monitor to record information about the registered interrupts. In DEVLORE, the monitor stores this information in the RMM-accessible realm memory and notifies the hypervisor that this interrupt arrived by writing to normal world hypervisor-accessible memory. Now, all that remains is to ensure that DEVLORE checks the monitor physical interrupt information to guarantee benign virtual interrupt delivery.

Delegate-but-check Virtual Interrupt Management. As in CCA, the hypervisor still manages interrupts for the CVMs in DEVLORE. So, the hypervisor should virtualize the device interrupts that the monitor trapped for the CVMs. To enable this, DEVLORE changes the hypervisor to first check if there is any pending interrupt from the monitor every time the hypervisor executes. If there are pending interrupts, the hypervisor injects the corresponding virtual interrupts into the CVM. According to CCA specifications, the hypervisor sends the vGIC configuration to the RMM. With DEVLORE, the RMM checks this configuration and then programs the vGIC to fire the virtual interrupts to the CVM, on CVM entry.

6.2 Checking Virtual Interrupts

Next, we explain checks that the RMM has to perform with the monitor cooperation. We start with simple checks that DEVLORE can do but will be insufficient and iterate over them to build towards our full set of checks. Fig 4 shows each set of checks with an example.

Check #1: Total Count. To begin with, assume that the RMM performs a simple count-based check when the hypervisor injects virtual interrupts into a CVM. Concretely, the RMM uses the physical interrupt information that the monitor stores to compute the total number of interrupts that should be injected for each CVM. When the hypervisor injects the virtual interrupts to the CVM, the RMM decrements the count of pending interrupts for that CVM. This solution stops the hypervisor from injecting too many or too few interrupts (see Fig 4.a). However, this method fails to prevent attacks where the hypervisor maliciously injects a virtual interrupt that the device had never sent.

Check #2: Origination from a Physical Interrupt. To stop

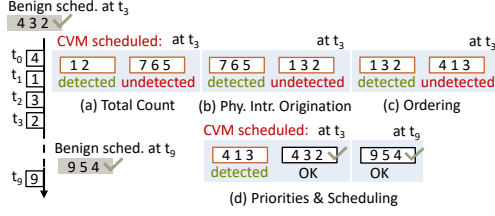


Figure 4: Device interrupts 1-9 are registered by a CVM for DEVLORE secure interrupt delivery. Interrupt number = interrupt priority. vGIC window size $n=3$. (a) Total Count: will allow any interrupt number [7,6,5] through as long as it matches the expected count=3. (b) Tie to Physical Interrupt: will block interrupts that do not have a corresponding physical interrupts [7,6,5], but will not respect the interrupt ordering [1,3,2]. (c) Ordering: we consider the ordering and detect violations blocking [1,3,2] but does not account for priorities and will allow [4,1,3] injection (d) Priorities: considers priorities and allows interrupts equivalent to benign scheduling.

the hypervisor from arbitrarily injecting virtual interrupts, the RMM should tie the authenticity of the physical interrupts to the virtual interrupts. For this, the RMM requires the monitor to write the registered physical interrupt numbers that arrive to a list in realm memory. Then, when the hypervisor injects the virtual interrupt, the RMM ensures that for each virtual interrupt programmed into the vGIC there is a corresponding entry in the monitor list and only then forwards the virtual interrupt. In CCA, when the hypervisor injects the virtual interrupt via the RMM, it can send n (where n depends on the GIC implementation) distinct interrupts to the vGIC (see Sec 2). So, for each virtual interrupt injection request, the RMM checks that all of the interrupts in the request are authentic. Tying the virtual interrupt to physical interrupts stops the hypervisor from arbitrarily injecting other interrupts but cannot ensure interrupt ordering (see Fig 4.b).

Check #3: Ordering. DEVLORE needs to store an ordered list of physical interrupts when they arrive at the monitor to guarantee that the hypervisor cannot reorder them. Then, the RMM can use this ordered list to check the virtual interrupts that the hypervisor wants to inject. We observe that because the hypervisor can inject n distinct interrupts at once to the CVM, DEVLORE does not need to guarantee ordering for these n distinct interrupts. If the RMM programs the vGIC with n distinct interrupts, when entering the CVM, the hypervisor will fill all n at the same time to the CVM. So the RMM uses the monitor ordered list to check ordering with a window size of n . This prevents the hypervisor from re-ordering the interrupts that are outside this window (Fig 4.c). However, this alone cannot achieve DEVLORE interrupt isolation. During benign operation, the hypervisor injects virtual interrupts based on the priority of all the pending interrupts.

	t_0	t_1	t_2	t_3
Benign				
pending int.	1 1 1 1	1 1 1 1	1 1 4 5	1 1
hyp. op.	schedule	schedule	schedule	schedule
virt. intr.	1	1	5 4	1
Benign				
pending int.	1 1 1 1	1 1 1 1	1 1 1 4 5	1 1 1 4 5
hyp. op.	schedule	stall	stall	schedule
virt. intr.	1			5 4 1
Malicious				
pending int.	1 1 1 1	1 1 1 1	1 1 1 4 5	1 1 1 4 5
hyp. op.	schedule	stall	stall	schedule
virt. intr.	1			5 4 1

Figure 5: Any malicious scheduling is always equivalent to some benign scheduling that the CVM expects.

Thus, time-based ordering cannot preserve the priority.

Check #4: Priorities. To check virtual interrupt priorities, DEVLORE requires the CVM to assign priorities to its registered device interrupts during device attach. The RMM uses this information to create per-CVM ordered priority lists of interrupts and uses it to check the virtual interrupt injections from the hypervisor (Fig 4.d). This check ensures that the hypervisor always injects the highest priority pending interrupts. We conclude that this check is sufficient to achieve DEVLORE guarantee that the interrupt behavior for registered device interrupts will be equivalent to a benign behavior (Fig 4.d). This check ensures that the hypervisor cannot inject or reorder interrupts in a way that a benign CVM execution does not expect. This guarantee is true even if the hypervisor maliciously delays CVM scheduling until a high-priority interrupt arrives to effectively never inject a lower-priority interrupt. We observe that a benign hypervisor provides no scheduling guarantees to a CVM, and expects the CVM to be able to handle any stalls in its scheduling (Fig 5 and Fig 4.d).

In summary, DEVLORE achieves interrupt isolation by enforcing the composition of checks #2, #3, and #4.

6.3 Acknowledging Interrupts

The above DEVLORE checks ensure isolated interrupt configuration and delivery. Despite these checks, the hypervisor still needs to acknowledge interrupts by writing to the memory-mapped registers for functionality. DEVLORE marks the memory-mapped register for acknowledgment as root memory, thus revoking the hypervisor access and forcing SMC. When the hypervisor tries to acknowledge any interrupt using a monitor call (SMC), the monitor first checks if this interrupt is isolated by DEVLORE. If not, the monitor performs the acknowledgment. On the other hand, for interrupts that DEVLORE isolates, it has to ensure that they are acknowledged appropriately for preserving the hypervisor functionality (Sec A.1).

Put together, DEVLORE mechanisms for blocking malicious physical injection to establish interrupt authenticity, recording authentic physical interrupts to create the ground

truth, and rigorous checking of virtual interrupt injections against the ground truth ensure interrupt isolation for CVMs.

7 Security Analysis

We argue why DEVLORE is secure against different adversaries. Prior works provide the guarantees listed in Sec 3 for device lifecycle management and memory isolation. Some of the attacks we detail here, are mitigated by these guarantees, but we explain them here for completeness, as without them DEVLORE guarantees may not hold.

Attach/Detach Device. The untrusted hypervisor can try to map a device to a physical address space it controls before attaching it to a CVM. This is not possible as the physical addresses that the devices are mapped to are fixed for the hardware platform and cannot be changed by software (G_{id}). Next, a hypervisor can try to emulate devices or assign the wrong device to a CVM which is stopped by ensuring unforgeable device identities (G_{id}). Concretely, DEVLORE checks the physical addresses that the hypervisor maps for the device using trusted platform information in the monitor to detect and stop such attacks. The hypervisor can also try to assign a device in a state that compromises the CVM when it is attached to it. To stop such attacks, DEVLORE soft resets the device before attaching it to the CVM (G_a). Similarly, the hypervisor can request DEVLORE to attach a device to a CVM when the CVM does not expect to have a device; this is stopped by G_a . Specifically, DEVLORE does not allow such requests and always expects the CVM to request a device attach. Finally, the hypervisor can try to detach a device while a CVM is using it to leak data from the device; this is mitigated by G_d .

Co-Resident CVMs. Attacker-controlled co-resident CVMs can try to directly access the device MMIO or DMA memory. An attacker can also try to set up realm memory regions overlapping with device memory of the victim CVM, but this would violate G_{mem} . DEVLORE adds all device memory to the CVM. When memory is added to a CVM, CCA ensures that this memory only belongs to one CVM and isolates it by programming the S2 tables in the RMM. Therefore, other CVMs will not have valid mappings for the device memory and cannot access it. An attacker-controlled CVM can try to inject interrupts to other CVMs. This is not possible as each CVM can only program its local vGIC and cannot access the vGIC of other CVMs, because the RMM does not expose any APIs to allow this.

SMMU Configuration and Attacker-controlled Devices. Hypervisor can try to change SMMU S2 table mappings to allow attacker-controlled devices access to victim device memory but this would violate G_{mem} . To stop these attacks and guarantee G_{mem} , DEVLORE protects the SMMU configuration in the root world to check and stop these attempts. Malicious devices can try to directly access victim device memory, but are stopped by G_{mem} . Concretely, DEVLORE ensures that the attacker-controlled device S2 tables will not have valid

mappings to access victim device memory. An attacker can use a device to fake victim device interrupts. But the hardware integration of the device on the platform is trusted and is part of CCA attestation report. Each device interrupt line is unique and tied to the hardware, so software attackers cannot use devices to fake other device interrupts.

GIC Configuration and Interrupt Injection Attacks. The hypervisor can try to reprogram the GIC to not trap to the monitor, compromising DEVLORE physical interrupt authentication mechanisms. Similarly, the hypervisor can try to reconfigure the interrupt priorities or acknowledge interrupts directly in the GIC to compromise DEVLORE device interrupt security guarantees. DEVLORE protects the GIC configuration in root memory and checks all hypervisor accesses to the configuration stopping such attempts. The hypervisor can mount Iago attacks on DEVLORE interfaces by invoking the monitor GIC configuration interface (see Fig 3) with arbitrary addresses in the root world to read or write from [23]. DEVLORE stops such attempts by checking that the address range that the hypervisor requests to operate on using this interface is in the GIC configuration space. Besides the memory-mapped GIC configuration, the Arm architecture contains system registers for GIC configuration. The hypervisor can try to use these registers to compromise DEVLORE interrupt isolation. However, DEVLORE traps all registered interrupts to EL3 by marking them as EL3 interrupts (i.e., Group 0 interrupts). On Arm, the hypervisor executing in normal world EL2 cannot use the system registers to configure these EL3 interrupts, thus stopping such attacks.

The hypervisor can stall CVM scheduling (Fig 5) and wait for a high priority interrupt to arrive. It can use this malicious scheduling to not inject lower priority interrupts that had arrived earlier. However, we observe that this behavior could also occur during CVM execution with a benign hypervisor as hypervisors do not provide scheduling guarantees for CVMs. Thus, this malicious stalling of the CVM is safe and does not lead to an interrupt injection behavior that diverges from what the CVM normally expects. The hypervisor can try to inject an interrupt multiple times (e.g., 16 times) with one vGIC programming request. This is not possible, as the vGIC injects any given interrupt only once on every realm entry. Therefore, while the vGIC can be programmed to fire n distinct interrupts at once, it cannot be programmed to fire the same interrupt multiple times at once. Finally the hypervisor can try to interrupt the RMM while it performs checks; but the RMM masks all interrupts while it executes.

8 Implementation

To prototype DEVLORE on the FVP, change 3054 LoC in the trusted firmware v2.8 (TF-A) which we use as the monitor [7], 1188 LoC in RMM v0.2.0 [14], and 2706 LoC the Linux kernel v6.2.0 (cca-full/rfc-v1) [53] which we use as the host and guest OS (guest: 60 LoC, host: 2646 LoC). The Arm

Table 1: DEVLORE new and existing API for device lifecycle management, memory isolation, and interrupt isolation.

API	Status	Description
rsi_attach_dev	New	Request attach device to CVM
rsi_detach_dev	New	Request detach device from CVM
rmi_dev_finalize	New	Hyp. finished adding device memory to realm
rmi_data_create	Existing	Update SMMU if devices attached to CVM
rmi_data_destroy	Existing	Update SMMU if devices attached to CVM
rmi_granule_delegate	Existing	update 2 GPTs
rmi_granule_undelagate	Existing	update 2 GPTs
smc_ack_int	New	Acknowledge interrupt
smc_gic_config	New	R/W to GIC configuration
smc_prot_int	New	Mark interrupts as protected

reference implementation uses KVM in the Linux kernel and a virtual machine manager (VMM) called kvmtool to build and deploy CVMs which we reuse. We change VMM with 590 LoC to add device support. Tab 1 shows DEVLORE interface changes.

Changes to Platform Boot. Like previous works, we create an additional GPT during boot for the device-side GPT (GPT_d) to allow devices access to CVM memory (Sec A.3). To ensure memory isolation, we move the SMMU configuration and memory to the root world [61, 67]. We also move GIC configuration to the root world during boot. Further, we modify the RMM to store device information for the platform (i.e., interrupt to device mappings). During runtime, the RMM looks up this information and uses it to check when the CVMs register interrupts.

Attach/Detach. Kvmtool does not have integrated device support that we can use in CCA, so we first add it and use it as our baseline (Sec A.2). Next, we use this, to implement mechanisms to attach, detach, manage integrated devices, and perform interrupt isolation. First, CCA does not allow devices direct access to CVM memory which they need to communicate with CVMs. To enable this, like previous works, we create two views by programming two different GPTs: GPT_c for the cores, and GPT_d for the devices (i.e., the SMMU GPT) as shown in Sec A.3 [61]. In GPT_d , we present the CVM memory as normal world to the device while in GPT_c we present the same memory as realm world.

To allow CVMs to register specific device interrupts we implement a new RSI call (`rsi_attach_dev`) in the guest Linux kernel and the RMM, which the CVM invokes with the device details (i.e., ID, memory-mapped GPAs, registered device interrupts). The RMM checks this request by looking up platform information (e.g., physical address ranges for device MMIO, device interrupt numbers) and then forwards the request to the hypervisor. Then, the hypervisor transitions all memory for the device to realm world (`rmi_granule_delegate`), adds this memory to the CVM (`rmi_data_create`), and invokes a new `rmi_dev_finalize` call. In response, the RMM checks that the hypervisor has installed the correct mappings for the CVM based on the CVM request. For memory isolation, the RMM invokes the monitor to configure the SMMU and marks the CVM mem-

ory as normal world in GPT_d and soft resets the device. Then, for registered interrupts, the RMM invokes the monitor (`smc_prot_int`) to configure it and returns to the CVM indicating that the device attach is complete. The CVM can request a device detach (`rsi_detach_dev`).

Interrupt Configuration. To isolate physical interrupts, we move the GIC memory-mapped configuration space to root. So, all hypervisor GIC configurations cause Granule Protection Fault (GPF) (Steps 1–2 in Fig 3). We modify the GPF handler to invoke the monitor (`smc_gic_config`) to check and perform the configuration (Steps 3–4 in Fig 3). We implement an SMC (`smc_prot_int`) for the RMM to invoke for registering interrupts for isolation with DEVLORE to ensure that they always trap to EL3. To do this, in the monitor, we first configure the GIC to trap all Group 0 interrupts to EL3 and program the GIC entry for the registered interrupts as belonging to Group 0. This process ensures that these interrupts always trap to EL3 (Step 1 in Fig 2.b).

Interrupt Delivery. We allocate a shared data structure for the RMM (realm) and hypervisor (normal) to track the pending protected interrupts. The monitor populates these data structures for each registered interrupt trap (Steps 1-3 in Fig 2.b) and notifies KVM using a software generated interrupt (Step 4, 5). KVM injects the pending interrupts to the CVM (Step 6) by requesting vGIC system register programming, which the RMM checks (Step 7) before forwarding them to the CVM (Steps 8–9).

Interrupt Acknowledgment. If possible, (i.e., without causing an interrupt storm), the monitor acknowledges any registered interrupts immediately before invoking KVM (Steps 1-5). Otherwise, after the virtual interrupt fires in the guest (Step 10) and the guest handler completes, the RMM traps the guest and invokes the monitor (`smc_ack_int`) to acknowledge the corresponding physical interrupt (Sec 6.3 and Sec A.1).

Performance Prototype. Given that the FVP is not cycle accurate, we approximate DEVLORE performance with OpenCCA [18]. OpenCCA runs on the Rock5b platform (see Sec 9.3), where the GPU is not connected to an SMMU. To enable GPU passthrough with DEVLORE, we modify the hypervisor to enforce a 1:1 guest-physical-to-host-physical memory mapping for CVM memory. This allows the GPU to DMA to guest memory. OpenCCA further lacks proper GPU driver support, so we port kbase [9] driver to Linux 6.12. Overall, we change ≈ 500 LoC LoC in OpenCCA.

9 Evaluation

We evaluate DEVLORE interrupt protection mechanism using both a functional and a performance prototype. The functional prototype validates DEVLORE compatibility with Armv9 and Arm CCA, but is limited to software simulation; the performance prototype evaluates DEVLORE performance impact on Armv8 hardware as an estimate. The two prototypes balance

simulation-based validation with hardware-based approximation to bridge the current lack of publicly available CCA platforms. Like previous works [18, 57, 61, 67, 76], our evaluation therefore provides a best-effort estimate of DEVLORE functionality and overheads on future CCA-based systems.

Research Questions. We aim to answer 5 evaluation questions:

Q1: Drivers & Protection: To what extent is DEVLORE compatible with existing device drivers, and does it correctly enforce interrupt protection?

Q2: Impact on System Operations: How does DEVLORE affect platform boot and non-interrupt workloads?

Q3: Interrupt Flow Breakdown: What is the runtime cost per protected interrupt?

Q4: Sustained Interrupt Load: How does DEVLORE perform under high-frequency interrupt stress tests?

Q5: Case Studies Can DEVLORE support unmodified real-world apps on both software simulation and hardware?

9.1 Evaluation Configurations

To evaluate DEVLORE on **Q1** to **Q5**, we first define two evaluation configurations: (1) **Baseline (B)**; execution on the CCA software stack without interrupt protection, and (2) **Interrupt Isolation (I)**; which integrates DEVLORE interrupt protection into the same stack. For both, we boot the platform, start a realm world CVM, attach integrated devices, and measure interrupt-related overheads.

9.2 Functionality (Q1)

We validate DEVLORE functionality and its compatibility with software simulation. For this, we evaluate our prototype on Arm Fixed Virtual Platform (FVP) version FVP_Base RevC 2xAEMvA 11.20_15 Linux64, which models Arm AEM cores with Realm Management Extension (RME) support. We configure the platform with one cluster of four AEMvA cores and 3 GB of RAM.

Q1: Drivers & Protection. The FVP is a functional simulator without accurate timing or microarchitectural behavior. We refrain from performance measurements on this platform [18]. Instead, Tab 2 and Sec B.1 demonstrate DEVLORE compatibility with unmodified device drivers across the implemented set of devices on the FVP, including GPU and UART. For these compatibility tests, we attach four devices to the CVM using DEVLORE and run their respective drivers. We then validate that DEVLORE enforces correct device isolation with 4 checks. These checks instantiate adversarial behaviors from Sec 7 and empirically validate that our implementation enforces DEVLORE security guarantees (details in Sec B.2).

Table 2: Device-driver compatibility on the Arm FVP. DEVLORE supports unmodified drivers in CVM for all tested peripherals. IRQ: Driver uses interrupts. LED and Button do not generate interrupts, thus do not need interrupt protection.

Device	IRQs	Driver Mod.	Protected w/ Dia	Setup & Benchmark
GPU	✓	No	✓	G76 kbase driver Midgard test suite
UART	✓	No	✓	Amba-pl011 driver (nano, browser, irc)
PS2/Mouse & Keyboard	✓	No	✓	Amba-kmi driver (move mouse, type n keys)
LED & Button	✗	No	N/A	Read button state, flash LED

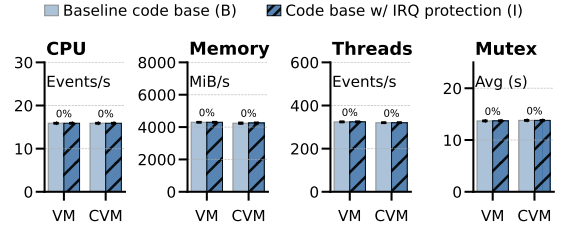


Figure 6: Sysbench [39] on Arm board (**Q2**) in a normal world VM (VM), and CVM for baseline configuration (**B**) and codebase with DEVLORE protection (**I**).

First, we check that all benign interrupts that device workloads (Tab 2) raise, are always delivered to the CVM as expected. Next, we test that 10k malicious virtual interrupts from the hypervisor are always rejected by the RMM with DEVLORE. This confirms that the RMM only delivers interrupts backed by a recorded physical interrupt. Further, we try to inject malicious physical interrupts from the hypervisor by writing to the GIC configuration and report that all such accesses trap to the monitor and are blocked. Finally, we use the FVP SMMU test engine to model a malicious device trying to access MMIO region of a CVM device. Our tests show that our implementation blocks such malicious accesses and reports GPT violations. Similarly, we see that when the hypervisor tries to access the MMIO memory, they are blocked and raise GPT faults.

Taken together, these experiments exercise the main attack vectors discussed in Sec 7 and show that DEVLORE enforces interrupt isolation under a malicious hypervisor and untrusted device.

9.3 Performance (Q2-Q4)

We approximate DEVLORE performance on Armv8 hardware using the Rock5B platform. Next, we detail our measurement setup on an Armv8 board before answering **Q2** to **Q4**.

All measurements run on OpenCCA, an open-source framework of Arm CCA [18]. We prototype on commit version 07fcc51 [19] and run DEVLORE on a RockChip Rock5b platform. The Rock5b is powered by an RK3588 SoC with

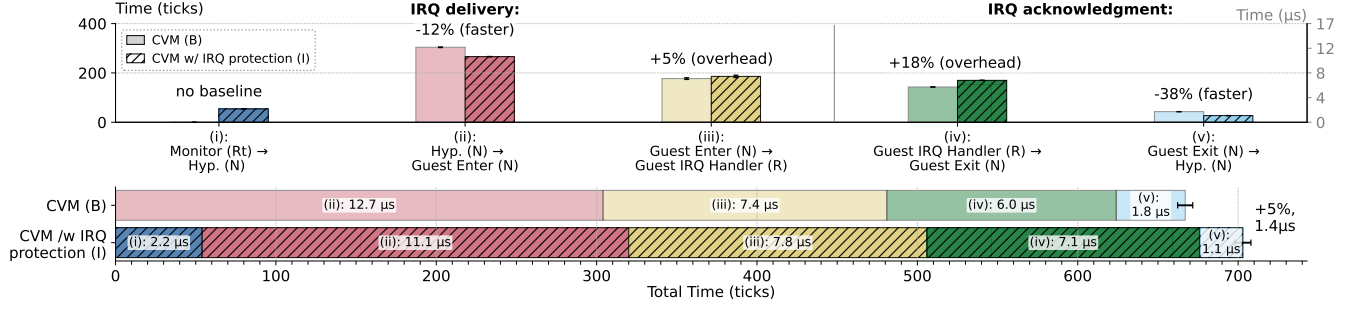


Figure 7: Interrupt flow breakdown (i) - (v) for GPU under DEVLORE on Arm board (Q3). Both plots (top and bottom) show the same experiment, comparing a CVM with (I) and without (B) interrupt protection. Top Plot: Time per stage of interrupt flow: (i) monitor receives physical interrupt in DEVLORE and notifies hypervisor (Steps 1-5 in Fig 2); (ii) Hypervisor schedules the guest (Step 6 in Fig 2); (iii) Guest enters and runs its IRQ handler (Steps 7-10 in Fig 2); (iv) Guest handles interrupt and exits to hypervisor; (v) Hypervisor processes end of interrupt acknowledgement. Bars show stage time for B (solid) and I (hatched). Percentages above indicate relative overhead of I w.r.t B. No baseline data (B) exists for (i) because IRQ directly lands in (ii). DEVLORE is slower in stages (i), (iii) and (iv) due to interrupt authenticity checks in monitor and RMM, but faster in stages (ii) and (v) because part of hypervisor bookkeeping is shifted into earlier stages. Interrupt transitions into different worlds: N: Normal world, Rt: Root world, R: Realm world. Bottom Plot: Total end-to-end interrupt handler time from (i) - (v). Smaller is better. 1 tick = 41.67 ns.

16 GB of RAM and a Mali G610 GPU. We follow the experimental setup of OpenCCA and boot the platform on 4 Cortex A55 cores, and fix CPU frequency to 1.8 GHz using `userspace` governor. We apply DEVLORE changes on top of the OpenCCA tree (TF-A v2.11, RMM v0.5, Linux v6.12), and boot both a normal world VM and CVM with `kvmtool`. We measure latency using Counter-timer Physical Count register (CNTPCT_EL0).

Q2: Platform Boot. We first evaluate the impact of DEVLORE on platform boot. For this, we boot the hypervisor on the 4 Cortex-A55 cores and compare the total boot time of the baseline (B) and DEVLORE software changes (I). B executes hypervisor boot in 8.9s compared to 9.1s for I. This leads to a total platform boot overhead of 2.4% or 221 ms. 80% of this overhead (+177.7ms) is because of SMMU setup for memory isolation. DEVLORE GIC configuration only adds 0.05% (+0.1ms) overhead during stage-3 bootloader (BL31).

Q2: Hypervisor GIC Traps. As GIC protection forces all hypervisor GIC accesses to trap into the monitor, we now examine the number and cost of these traps during Linux kernel boot. During platform boot with DEVLORE code changes (I), we observe 1816 traps (445 reads, 1371 writes) to the monitor to configure the GIC (with 999 kernel modules and 7262 driver object files compiled in-tree). Each trap takes 2.5μs on average. For context, the entire Linux kernel boot in I takes 4.8s, which is several orders of magnitude larger than a single trap.

Q2: Impact on Normal Operations. As we have detailed in Sec 8, DEVLORE protects the GIC and SMMU in the root world. This requires the hypervisor to call the monitor for any changes to SMMU and GIC configuration. SMMU protection

only adds to CVM boot cost (during memory delegation), whereas GIC protection affects even normal world runtime. To show the effect of DEVLORE on non-interrupt workloads, we run `sysbench` [39] on a normal world VM, and CVM for both our baselines B and I, as shown in Fig 6. As expected, DEVLORE has no impact on non-interrupt workloads.

Q3: Interrupt Flow Breakdown. Recall that DEVLORE changes the interrupt flow for protected interrupts. We quantify this cost by measuring the time from when the device raises a physical interrupt until the hypervisor processes the guest’s end-of-interrupt acknowledgement. We break this path into interrupt delivery ((i) - (iii)), and interrupt acknowledgement ((iv) - (v)). We use the Mali G610 GPU on the Rock5B, which generates frequent high-rate interrupts (Fig 8) and employs a complex driver stack (90k LoC [9]). We attach the GPU to a CVM and compare the baseline configuration (B) to DEVLORE interrupt protection (I). Averaged over 1000 interrupts, the end-to-end interrupt delivery & acknowledgement time in B is 669 ticks (27.9 μs, 95% CI: ±0.2 μs), while DEVLORE increases this to 703 ticks (29.3 μs, CI: ±0.2 μs), a 5% overhead (Fig 7, bottom).

The stage-wise breakdown in Fig 7 (top) shows that DEVLORE is slower when the protected interrupt is first raised and recorded in the monitor ((i), +2.2 μs), during authenticity checks in the RMM on guest entry ((iii), 5%, +0.4 μs), and guest exit for end-of-interrupt acknowledgement in the monitor ((iv), +18%, +1.1 μs). In contrast, DEVLORE is faster when the hypervisor schedules the guest with the new virtual interrupt (stage (ii), -12%, -1.6 μs), and when it completes post end-of-interrupt bookkeeping ((v), -38%, -0.7 μs). This is because, with DEVLORE, the monitor manages the physical in-

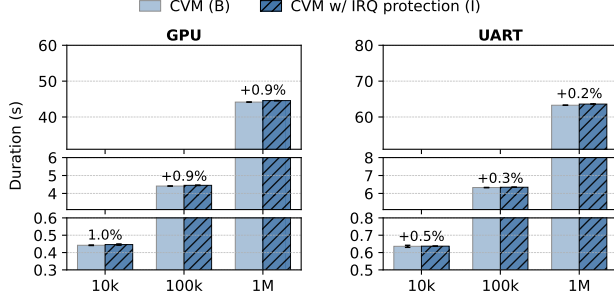


Figure 8: Interrupt Stress Test (Q4) for GPU (Midgard ktuft) and UART device on Arm board for CVM (B and I). X-Axis: # of IRQ. Y-Axis: duration in seconds. Smaller is faster.

terrupt state (disable on trap, re-enable on acknowledgement) rather than leaving this work to the hypervisor. In summary, recording physical interrupts and checking virtual interrupts contribute predominantly to the 5% overhead that DEVLORE adds to the end-to-end interrupt latency.

Q4: Sustained Interrupt Load. We examine how these overheads manifest under continuous device activity. For this, we run an interrupt stress test using both the GPU and UART device. We attach each device to their own CVM and configure the device to generate a high frequency of interrupts. For GPU, we use Mali Ktuft Midgard benchmark suite [9]. For UART, we modify the Synopsys DesignWare 8250 driver to run in internal loopback mode. This way, it raises a physical interrupt on each write to the transmit register. Fig 8 shows the total runtime to process interrupts (X-Axis) for both devices in a CVM, using 10k, 100k and 1M interrupts (B and I). Under continuous load, the overhead stays within 1% with small run-to-run variations (stdev below 1.2% of mean, N=25, 100, 200 for 1M, 100k, 10k IRQ). This indicates that per-interrupt costs do not accumulate at scale.

9.4 Case Studies (Q5)

Finally, we validate DEVLORE with real-world apps on the FVP and board.

Q5: Case Studies on FVP. Our case studies on the FVP showcase DEVLORE compatibility real-world apps. All apps run unmodified within a protected CVM. We chose three applications that use the UART device: terminal browser, IRC client, and text editor. We cannot use the FVP to estimate DEVLORE overheads as we cannot ensure consistent user actions across several runs. Instead, our videos showcase that we can interact with these applications. This demonstrates DEVLORE compatibility and usability compared to executing them in the realm world without interrupt protection. The two videos: [2] for B and [1] for I, boot a CVM on the FVP, attach the integrated UART device, and interact with the apps (details in Sec B.3).

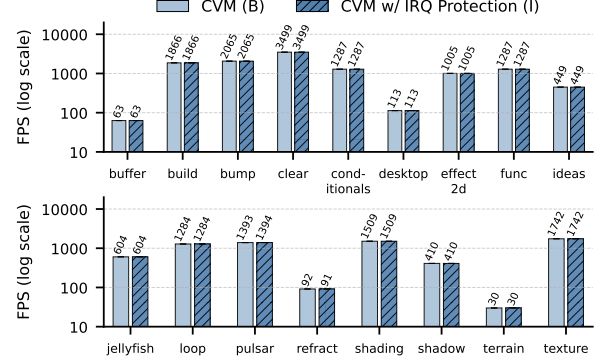


Figure 9: GImark2 on Arm Board (Q5). X-Axis: gImark2 benchmarks for realm world CVM (B) and realm world CVM with DEVLORE interrupt protection (I). Y-Axis: GPU frames per second in log scale.

Q5: Case Study on Board. We evaluate our design under realistic GPU workloads using gImark2 [3], a suite of userspace workloads that capture the mixed compute, memory, and driver interactions of typical integrated GPU apps. The workloads generate 2.5k - 32k GPU interrupts (exact configuration in Sec B.4). We cross-compile version 2023.01, attach the G610 GPU to the CVM, and run all benchmarks unmodified in their standard configuration in a CVM and a CVM with DEVLORE. Enabling DEVLORE interrupt protection, across all 17 benchmarks, has no measurable impact on the end-to-end throughput (see Fig 9). Frame rates for CVM with and without DEVLORE are almost identical. The geometric mean ratio across all benchmarks is +0.06%. So, DEVLORE interrupt protections does not add measurable overhead beyond the baseline realm cost, indicating no end-to-end performance overheads for CVMs.

10 Related Work

Arm CCA. Among the works on Arm CCA [22, 24, 25, 33, 45, 57, 61, 67, 73, 76–78], many offer better abstractions [22, 24, 25, 78] or enable new security guarantees [73, 77]. We discuss the works related to DEVLORE. Shelter isolates userspace applications in the normal world using multiple GPTs [76]. Cage and Portal use several GPTs to isolate mutually distrusting GPU computation [57, 67]. Both Shelter and Cage reprogram GPCs in the cores and SMMU during runtime and as a result, incur context switch overheads because of GPC TLB flushes. In contrast, DEVLORE uses techniques from Acai which uses two GPTs and does not reprogram the GPCs during runtime [61]. Ongoing efforts to verify the RMM and the CCA firmware [33, 45] can be extended to DEVLORE TCB.

Device Protection in Arm. Cage and Strongbox build sup-

port, for CCA CVMs and TrustZone trusted apps respectively, to securely compute on integrated Arm GPUs—without trusting the GPU driver and runtime to reduce the TCB [27, 67]. DEVLORE does not aim to reduce the TCB and includes the device drivers and runtimes in the TCB of the CVMs. In contrast, DEVLORE like Portal supports attaching a wide array of integrated devices to CVMs [57]. Similarly, Acai connects TEE-enabled PCIe accelerators to CVMs [61]. Arm hardware extension for Device Assignment (RME-DA) allows TEE-enabled PCIe accelerators to directly access CVM memory [12]. Although Acai and RME-DA are not applicable to integrated devices, DEVLORE reuses techniques from Portal, Acai, and RME-DA to manage the device life-cycle and ensure memory isolation. As a new contribution, DEVLORE ensures interrupt isolation for integrated devices with Arm CCA and extends these previous works on memory isolation to enable interrupt isolation. TrustZone has been repurposed to enable protected access to integrated devices [34, 35, 42–44, 62, 63, 74]. However, these works do not reason about VMs and hence virtual interrupt delivery. DEVLORE uses CCA which supports VM abstraction and thus reasons about multiplexing interrupts across realm VMs.

Integrated Device Protection in Other Architectures. On RISC-V, prior works attach devices to enclaves with hardware changes [17, 60]. DEVLORE uses Arm CCA, which has analogous hardware-based isolation [17]. Other works allow trusted hardware to create different views of memory like GPTs in CCA [20, 32, 40] or hardware-domains [72]. Intel SGX is used to perform trusted IO with SGX enclaves [28, 31, 38, 46, 69]. Instead of protecting user-space enclaves against malicious non-device interrupts [21, 26], DEVLORE isolates device interrupts for CVMs.

Interrupt Attacks. Side-channel attacks exploit unprotected timers, page-faults, IPIs [36, 54, 55, 64–66, 68, 71], and fake CPU exceptions, system calls, and exploit race conditions [41, 58, 59]. These interrupts are raised by the processor (e.g., page-faults, system calls, IPIs), whereas DEVLORE focuses on device interrupts. Prior works have studied how observing statistical information about integrated device interrupts (e.g., interrupt timing, number of interrupts) on Android can compromise confidentiality [29].

Interrupt Isolation on Arm. For *interrupt configuration*, prior works on TrustZone [27, 34, 42] isolate integrated device interrupts by marking the GIC configuration as secure world. However, in DEVLORE, we do not trust the secure world which can directly program the GIC to maliciously fire interrupts. So, departing from these works and for security in the CCA setting, DEVLORE marks GIC configuration as root. On the other hand, for *interrupt delivery*, like the above prior works, DEVLORE also marks interrupts as belonging to Group 0 (i.e., secure world) to trap them in EL3 [27, 34, 42]. Recent works on CCA need interrupt isolation for one interrupt ID (e.g., GPU or timer) and resort to the same mechanisms as works on TrustZone [67, 75] i.e., marking the interrupt as

secure world Group. Thus, they have to trust the secure world, which DEVLORE avoids. Other recent work on Arm CCA has proposed delegating interrupt management to the RMM for performance when defending against a side-channel adversary [22]. In their design, they pin CVMs to cores and to reduce the number of context switches to the hypervisor by using the RMM to manage the CVM timer interrupts. This is equivalent in security to the existing CCA threat model that does block malicious interrupt injection from the hypervisor. As the hypervisor still manages all other interrupts, their design would need DEVLORE interrupt isolation. DEVLORE is not specific to one interrupt ID, instead isolates device interrupts across multiple devices and CVMs thus addressing the gap in the CCA threat model.

TDISP, Intel TDX, and AMD SEV-SNP. TDISP is a PCIe feature that allows direct attachment of TEE-enabled devices to CVMs and does not reason about device interrupts. TDX-Connect and SEV-TIO are TDISP compliant implementations specific to Intel and AMD respectively. They protect CVM-bound interrupts by deploying trusted firmware and software to check virtual interrupt injections [5, 6, 37]. But, like CCA, they only allow or deny all device interrupts without granular interrupt isolation. DEVLORE insights can be applied to these other architectures, but with careful analysis.

11 Conclusion

In this paper, we present DEVLORE, a device interrupt isolation mechanism for CVMs. Our delegate-but-check strategy for interrupt isolation, allows the hypervisor to manage the GIC but strictly monitors the interrupt life cycle (configuration, delivery, acknowledgment). Our evaluation demonstrates this on the Arm FVP and an Arm Rock 5B Board.

References

- [1] DEVLORE: Screen Recording of Interactive Case Studies CVM with Interrupt Protection (I), 2025.
- [2] DEVLORE: Screen Recording of Interactive Case Studies CVM Baseline (B), 2025.
- [3] Ben Smith Alexandros Frantzis, Jesse Barker. *OpenGL 2.0 and ES 2.0 benchmark*, v2023.01, 2010-2025.
- [4] Chris Allegretta and Benno Schulenberg. *GNU nano text editor*, 1999-2025.
- [5] AMD. *AMD SEV-SNP: Strengthening VM Isolation with Integrity protection and more*, 2020.
- [6] AMD. *Linux SVSM (Secure VM Service Module)*, 2022-2024.
- [7] Arm. *Arm Trusted Firmware-A Documentation*, 2013-2025.

- [8] Arm. [NoMali: A simple Mali 6xx/7xx register interface model that doesn't do any rendering.](#), 2015-2016.
- [9] Arm. [Bifrost: Open Source Kernel Drivers](#), 2016-2025.
- [10] Arm. [Fast Models Fixed Virtual Platforms \(FVP\), Reference Guide](#), 2017-2023.
- [11] Arm. [SMMUv3TestEngine, Fast Models Fixed Virtual Platforms \(FVP\) Reference Guide](#), 2017-2024.
- [12] Arm. [Arm Realm Management Extension \(RME\) System Architecture](#), 2021-2023.
- [13] Arm. [Arm Confidential Compute Architecture \(ARM-CCA\)](#), 2022-2025.
- [14] Arm. [Trusted Firmware Implementation of the Realm Management Monitor \(RMM\)](#), 2022-2025.
- [15] Arm. [Powering Microsoft's Azure Cobalt 200 with Arm Neoverse CSS V3: The next generation of Arm-based compute for the AI era](#), 2025.
- [16] Azure. [Announcing Cobalt 200: Azure's next cloud-native CPU](#), 2025.
- [17] Raad Bahmani, Ferdinand Brasser, Ghada Dessouky, Patrick Jauernig, Matthias Klimmek, Ahmad-Reza Sadeghi, and Emmanuel Stempf. CURE: A Security Architecture with Customizable and Resilient Enclaves. In *USENIX Security Symposium*, pages 1073–1090, 2021.
- [18] Andrin Bertschi and Shweta Shinde. OpenCCA: An open framework to enable arm cca research. In *8th Workshop on System Software for Trusted Execution (SysTEX 2025)*, 2025.
- [19] Andrin Bertschi and Shweta Shinde. OpenCCA version (SysTEX 2025), Manifest commit 07fcc51, March 2025.
- [20] Ferdinand Brasser, David Gens, Patrick Jauernig, Ahmad-Reza Sadeghi, and Emmanuel Stempf. Sanctuary: ARMing TrustZone with User-space Enclaves. In *NDSS*, 2019.
- [21] Matteo Busi, Job Noorman, Jo Van Bulck, Letterio Galletta, Pierpaolo Degano, Jan Tobias Mühlberg, and Frank Piessens. Securing Interruptible Enclaved Execution on Small Microprocessors. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 43(3):1–77, 2021.
- [22] Charly Castes and Andrew Baumann. Sharing is leaking: blocking transient-execution attacks with core-gapped confidential vms. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, pages 267–281, 2024.
- [23] Stephen Checkoway and Hovav Shacham. Iago attacks: why the system call API is a bad untrusted RPC interface. *ACM SIGARCH Computer Architecture News*, 41(1):253–264, 2013.
- [24] Jiahao Chen, Zeyu Mi, Yubin Xia, Haibing Guan, and Haibo Chen. CPC: Flexible, secure, and efficient CVM maintenance with confidential procedure calls. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 1065–1082, 2024.
- [25] Jiayun Chen, Qihang Zhou, Xiaolong Yan, Nan Jiang, Xiaoqi Jia, and Weijuan Zhang. CubeVisor: A Multi-realm Architecture Design for Running VM with ARM CCA. In *2024 Annual Computer Security Applications Conference (ACSAC)*, pages 1–13. IEEE, 2024.
- [26] Ruan De Clercq, Frank Piessens, Dries Schellekens, and Ingrid Verbauwhede. Secure interrupts on low-end microcontrollers. In *2014 IEEE 25th International Conference on Application-Specific Systems, Architectures and Processors*, pages 147–152. IEEE, 2014.
- [27] Yunjie Deng, Chenxu Wang, Shunchang Yu, Shiqing Liu, Zhenyu Ning, Kevin Leach, Jin Li, Shoumeng Yan, Zhengyu He, Jiannong Cao, et al. StrongBox: A GPU TEE on Arm Endpoints. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 769–783, 2022.
- [28] Aritra Dhar, Ivan Puddu, Kari Kostinen, and Srdjan Capkun. ProximiTEE: Hardened SGX Attestation by Proximity Verification. In *Proceedings of the Tenth ACM Conference on Data and Application Security and Privacy*, page 5–16, 2020.
- [29] Wenrui Diao, Xiangyu Liu, Zhou Li, and Kehuan Zhang. No Pardon for the Interruption: New Inference Attacks on Android Through Interrupt Timing Analysis. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 414–432, 2016.
- [30] Thomas Dickey. [LYNX – The Text Web-Browser](#), 1997-2025.
- [31] Saba Eskandarian, Jonathan Cogan, Sawyer Birnbaum, Peh Chang Wei Brandon, Dillon Franke, Forest Fraser, Gaspar Garcia Jr., Eric Gong, Hung T. Nguyen, Taresh K. Sethi, Vishal Subbiah, Michael Backes, Giancarlo Pellegrino, and Dan Boneh. Fidelius: Protecting User Secrets from Compromised Browsers. In *IEEE Symposium on Security and Privacy*, 2019.
- [32] Erhu Feng, Dahu Feng, Dong Du, Yubin Xia, Wenbin Zheng, Siqi Zhao, and Haibo Chen. sIOPMP: Scalable and Efficient I/O Protection for TEEs. In *Proceedings of the 29th ACM International Conference on Architectural*

Support for Programming Languages and Operating Systems, page 1061–1076, 2024.

- [33] Anthony CJ Fox, Gareth Stockwell, Shale Xiong, Hanno Becker, Dominic P Mulligan, Gustavo Petri, and Nathan Chong. A Verification Methodology for the Arm® Confidential Computing Architecture: From a Secure Specification to Safe Implementations. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):376–405, 2023.
- [34] Friederike Groschupp, Mark Kuhne, Moritz Schneider, Ivan Puddu, Shweta Shinde, and Srdjan Capkun. It’s TEEtime: A New Architecture Bringing Sovereignty to Smartphones. *arXiv preprint arXiv:2211.05206*, 2022.
- [35] Seung-Kyun Han and Jinsoo Jang. MyTEE: Own the Trusted Execution Environment on Embedded Devices. In *NDSS*, 2023.
- [36] Wenjian He, Wei Zhang, Sanjeev Das, and Yang Liu. SGXlinger: A New Side-channel Attack Vector Based on Interrupt Latency against Enclave Execution. In *2018 IEEE 36th International Conference on Computer Design (ICCD)*, pages 108–114. IEEE, 2018.
- [37] Intel. [Intel Trust Domain Extensions \(Intel TDX\)](#), 2023-2025.
- [38] Yeongjin Jang and Sejin Keem. SGX-USB: Secure USB I/O Path for Secure Enclaves. *Hawaii International Conference on System Sciences (HICSS)*, 2024.
- [39] Alexey Kopytov. [Sysbench, CPU: 200k Primes, Memory: 1GB \(64K Blocks\), Threads: 8, Mutex: 8](#), 2006-2025.
- [40] Shan-Chyun Ku. What’s in RISC-V IOPMP. RISC-V Forum: Security, 2021.
- [41] Yoochan Lee, Changwoo Min, and Byoungyoung Lee. ExpRace: Exploiting Kernel Races through Raising Interrupts. In *30th USENIX Security Symposium*, 2021.
- [42] Matthew Lentz, Rijurekha Sen, Peter Druschel, and Bobby Bhattacharjee. SeCloak: ARM Trustzone-Based Mobile Peripheral Control. In *Proceedings of the 16th Annual International Conference on Mobile Systems, Applications, and Services, MobiSys*, 18, 2018.
- [43] Wenhao Li, Shiyu Luo, Zhichuang Sun, Yubin Xia, Long Lu, Haibo Chen, Binyu Zang, and Haibing Guan. VButton: Practical Attestation of User-Driven Operations in Mobile Apps. In *Proceedings of the 16th Annual International Conference on Mobile Systems, Applications, and Services, MobiSys ’18*, page 28–40, New York, NY, USA, 2018. Association for Computing Machinery.
- [44] Wenhao Li, Mingyang Ma, Jinchun Han, Yubin Xia, Binyu Zang, Cheng-Kang Chu, and Tieyan Li. Building Trusted Path on Untrusted Device Drivers for Mobile Devices. In *Proceedings of 5th Asia-Pacific Workshop on Systems, APSys ’14*, 2014.
- [45] Xupeng Li, Xuheng Li, Christoffer Dall, Ronghui Gu, Jason Nieh, Yousuf Sait, and Gareth Stockwell. Design and Verification of the Arm Confidential Compute Architecture. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022.
- [46] Hongliang Liang, Mingyu Li, Yixiu Chen, Lin Jiang, Zhuosi Xie, and Tianqi Yang. Establishing Trusted I/O Paths for SGX Client Systems With Aurora. *IEEE Transactions on Information Forensics and Security*, 15:1589–1600, 2020.
- [47] Linux. [Linux WCNSS driver](#), 2016-2025.
- [48] Fredi Machado. [Simple cross-platform Console IRC Client](#), 2011-2018.
- [49] Microsoft. [Microsoft and NVIDIA partnership continues to deliver on the promise of AI](#), 2024.
- [50] Nvidia. [Confidential Compute on NVIDIA Hopper H100 \(v1.0\)](#), 2023.
- [51] Nvidia. [NVIDIA Puts Grace Blackwell on Every Desk and at Every AI Developer’s Fingertips](#), 2025.
- [52] Tomball Paul Vu. Method and Apparatus for Controlling Interrupt Storms, U.S. Patent 7,120,717 B2, Oct. 2006.
- [53] Suzuki Poulouse. [\[RFC\] Support for Arm CCA VMs on Linux](#), 2023.
- [54] Ivan Puddu, Moritz Schneider, Miro Haller, and Srdjan Capkun. Frontal attack: Leaking Control-Flow in SGX via the CPU frontend. In *30th USENIX Security Symposium*, 2021.
- [55] Fabian Rauscher and Daniel Gruss. Cross-Core Interrupt Detection: Exploiting User and Virtualized IPIs. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024.
- [56] Joel Rosdahl. [miniircd – A \(very\) simple Internet Relay Chat \(IRC\) server](#), 2011-2025.
- [57] Fan Sang, Jaehyuk Lee, Xiaokuan Zhang, and Taesoo Kim. PORTAL: Fast and Secure Device Access with Arm CCA for Modern Arm Mobile System-on-Chips (SoCs) . In *2025 IEEE Symposium on Security and Privacy (SP)*, 2025.

- [58] Benedict Schlüter, Supraja Sridhara, Andrin Bertschi, and Shweta Shinde. WeSee: Using Malicious #VC Interrupts to Break AMD SEV-SNP. In *IEEE S&P*, 2024.
- [59] Benedict Schlüter, Supraja Sridhara, Mark Kuhne, Andrin Bertschi, and Shweta Shinde. Heckler: Breaking Confidential VMs with Malicious Interrupts. In *USENIX Security*, 2024.
- [60] Moritz Schneider, Aritra Dhar, Ivan Puddu, Kari Kostianen, and Srdjan Čapkun. Composite Enclaves: Towards Disaggregated Trusted Execution. *IACR CHES*, 2022.
- [61] Supraja Sridhara, Andrin Bertschi, Benedict Schlüter, Mark Kuhne, Aliberti Aliberti, and Shweta Shinde. Acai: Protecting Accelerator Execution with Arm Confidential Computing Architecture. In *USENIX Security*, 2024.
- [62] He Sun, Kun Sun, Yuewu Wang, and Jiwu Jing. TrustOTP: Transforming Smartphones into Secure One-Time Password Tokens. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pages 976–988, 2015.
- [63] He Sun, Kun Sun, Yuewu Wang, Jiwu Jing, and Haining Wang. TrustICE: Hardware-Assisted Isolated Computing Environments on Mobile Devices. In *2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, pages 367–378. IEEE, 2015.
- [64] Jo Van Bulck, Frank Piessens, and Raoul Strackx. Nemesis: Studying Microarchitectural Timing Leaks in Rudimentary CPU Interrupt Logic. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018.
- [65] Jo Van Bulck, Nico Weichbrodt, Rüdiger Kapitza, Frank Piessens, and Raoul Strackx. Telling Your Secrets without Page Faults: Stealthy Page Table-Based Attacks on Enclaved Execution. In *26th USENIX Security Symposium*, pages 1041–1056, 2017.
- [66] Stephan Van Schaik, Alex Seto, Thomas Yurek, Adam Batori, Bader AlBassam, Daniel Genkin, Andrew Miller, Eyal Ronen, Yuval Yarom, and Christina Garman. SoK: SGX.Fail: How Stuff Gets eXposed. In *2024 IEEE Symposium on Security and Privacy*, pages 4143–4162, 2024.
- [67] Chenxu Wang, Fengwei Zhang, Yunjie Deng, Kevin Leach, Jiannong Cao, Zhenyu Ning, Shoumeng Yan, and Zhengyu He. CAGE: Complementing Arm CCA with GPU Extensions. In *Network and Distributed System Security (NDSS) Symposium*, 2024.
- [68] Wenhao Wang, Guoxing Chen, Xiaorui Pan, Yinqian Zhang, XiaoFeng Wang, Vincent Bindschaedler, Haixu Tang, and Carl A. Gunter. Leaky Cauldron on the Dark Land: Understanding Memory Side-Channel Hazards in SGX. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017.
- [69] Samuel Weiser and Mario Werner. SGXIO: Generic Trusted I/O Path for Intel SGX. In *Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy*, pages 261–268, 2017.
- [70] Wikipedia. [Today’s featured article](#), 2025.
- [71] Yuanzhong Xu, Weidong Cui, and Marcus Peinado. Controlled-Channel Attacks: Deterministic Side Channels for Untrusted Operating Systems. In *IEEE S&P*, 2015.
- [72] Zhihao Yao, Seyed Mohammadjavad Seyed Talebi, Mingyi Chen, Ardalan Amiri Sani, and Thomas Anderson. Minimizing a Smartphone’s TCB for Security-Critical Programs with Exclusively-Used, Physically-Isolated, Statically-Partitioned Hardware. In *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services*, pages 233–246, 2023.
- [73] Zhenyu Ye, Lei Zhou, Fengwei Zhang, Wenqiang Jin, Zhenyu Ning, Yupeng Hu, and Zheng Qin. FortifyPatch: Towards Tamper-Resistant Live Patching in Linux-Based Hypervisor. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 38–50, 2024.
- [74] Kailiang Ying, Amit Ahlawat, Bilal Alsharifi, Yuexin Jiang, Priyank Thavai, and Wenliang Du. TruZ-Droid: Integrating TrustZone with Mobile Operating System. In *Proceedings of the 16th Annual International Conference on Mobile Systems, Applications, and Services*, MobiSys 18, page 14–27, 2018.
- [75] Chuqi Zhang, Jun Zeng, Yiming Zhang, Adil Ahmad, Fengwei Zhang, Hai Jin, and Zhenkai Liang. The HitchHiker’s Guide to High-Assurance System Observability Protection with Efficient Permission Switches. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, page 3898–3912, 2024.
- [76] Yiming Zhang, Yuxin Hu, Zhenyu Ning, Fengwei Zhang, Xiapu Luo, Haoyang Huang, Shoumeng Yan, and Zhengyu He. SHELTER: Extending arm CCA with isolation in user space. In *32nd USENIX Security Symposium (USENIX Security 23)*, 2023.

- [77] Yiming Zhang, Fengwei Zhang, Xiapu Luo, Rui Hou, Xuhua Ding, Zhenkai Liang, Shoumeng Yan, Tao Wei, and Zhengyu He. SCRUTINIZER: Towards Secure Forensics on Compromised TrustZone. In *32nd Network and Distributed System Security Symposium (NDSS)*, 2025.
- [78] Qihang Zhou, Wenzhuo Cao, Xiaoqi Jia, Peng Liu, Shengzhi Zhang, Jiayun Chen, Shaowen Xu, and Zhenyu Song. RContainer: A Secure Container Architecture through Extending ARM CCA Hardware Primitives. In *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24-28, 2025*. The Internet Society, 2025.

A Implementation

A.1 Acknowledging Interrupts in DEVLORE

A benign hypervisor typically acknowledges physical interrupts as soon as it receives an interrupt and then injects a virtual interrupt to the VM i.e., before servicing the interrupt. This is to ensure progress in the system—if the acknowledgment is delayed for a particular interrupt, all further interrupt invocations for this id will be blocked. However, the benign hypervisor should not use the above greedy acknowledgment approach for special types of interrupts. Specifically, certain devices constantly re-trigger the same interrupt if an interrupt is acknowledged too early i.e., the handler in the VM has not performed its operations. For example, the keyboard writes pending characters to its own memory and sends an interrupt. However, if the physical interrupt is acknowledged before the VM handler consumes the characters the keyboard thinks that the interrupt was dropped and resends it. If the gap between the hypervisor’s greedy acknowledgment and the VM’s end of interrupt handler is large, the interrupts keep arriving at the GIC, which injects them into the unblocked cores one after the other. This is referred to as an interrupt storm [52].

The hypervisor knows the type of interrupt from the platform device information and uses this information to decide if it is safe to perform greedy acknowledgments for cases where it is safe. If it is not safe, the hypervisor first services the interrupt and injects a virtual interrupt to the VM. Then, the hypervisor waits for the virtual interrupt acknowledgment from the VM and only then performs a physical interrupt acknowledgment. So, when the hypervisor requests to acknowledge registered interrupts, DEVLORE relies on the device tree information. Based on the platform device information, DEVLORE either allows greedy acknowledgment or waits for the CVM to acknowledge the virtual interrupt and only then allows a physical interrupt acknowledgment.

Table 3: Arm CCA PAS access rules. The columns denote the security state of the processing element that originates the memory access request. The rows denote the different physical address spaces.

	Root	Realm	Normal	Secure
Root	Yes	No	No	No
Realm	Yes	Yes	No	No
Normal	Yes	Yes	Yes	Yes
Secure	Yes	No	No	No

A.2 Adding Device Support to kvmtool

To add device support to kvmtool, we first map the device physical memory to the VM in its S2 tables. With this, the VM can directly access and control the device. Next, we build a framework in kvmtool that uses KVM to trap on all physical device interrupts and program the vGIC to inject them as virtual interrupts into the VM. With this, the VM can directly receive interrupts from the device.

The only thing left is for KVM to decide when to acknowledge the interrupt i.e., perform the end-of-interrupt operation. Depending on the type of the interrupt, KVM can either acknowledge it greedily or has to wait for the VM to first acknowledge the interrupt (Sec A.1). If KVM prematurely acknowledges the latter type of interrupt, it could result in an interrupt storm. So, for interrupts which KVM has to wait for the VM to acknowledge, we first disable the interrupt in the GIC when it first arrives to prevent an interrupt storm. Then, we acknowledge the physical interrupt. To determine when to re-enable the interrupt again, we use KVM to check the vGIC state on every VM exit and only re-enable the physical device interrupt after the VM has acknowledged the corresponding virtual interrupts. We modify KVM to use the vGIC state to re-enable any physical device interrupts after the VM has acknowledged it. On the other hand, for interrupts that KVM can greedily acknowledge, KVM directly performs this operation before programming the vGIC.

A.3 Attaching Devices to CVMs

Tab 3 shows Arm CCA physical address space access rules. DEVLORE adapts the device attach process of prior works to allow CVMs to enable DEVLORE interrupt isolation [12, 57, 61]. As introduced in these works, DEVLORE uses 2 GPTs to attach devices to CVMs in DEVLORE. This is visualized in Fig 10.

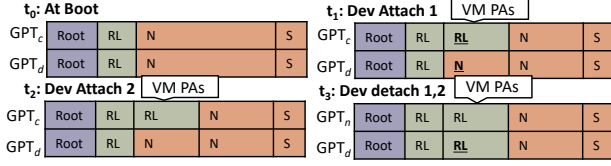


Figure 10: DEVLORE’s 2 GPTs. t₀: Create and populate 2 GPTs (GPT_c for cores, and GPT_d for devices). t₁: Update GPT_d on device attach to allow devices access to CVM memory by marking CVM as normal world (N) in GPT_d. t₂: No updates. t₃: Update GPT_d to revoke device access to CVM memory by marking it back to realm world (RL).

B Evaluation Details

B.1 Device Setups on FVP

We attach the following devices to the FVP to test DEVLORE compatibility with unmodified device drivers. The next paragraphs describe the devices and their workloads.

Because the FVP is not cycle accurate, and because asynchronous interrupts like timers can occur irregularly, neither cycle nor event counts are a reliable performance metric. We use FVP’s Model Trace Interface (MTI) to count events. As per our experimental measurements on the FVP, in the baseline **B**, guest entry and exit follow a 1:1 ratio: each RMI entry has a matching SMC exit invocation. With **I**, this changes because DEVLORE interrupt handling adds extra monitor calls: for each interrupt, the RMM invokes the root-world monitor to acknowledge the interrupt.

Keyboard. We use the FVP X11 visualization component that translates keyboard and mouse input on the x86 host to an emulated PL050 PS/2 peripheral on the FVP. We eliminate typing speed effects by using an x86 helper application that uses the X11/Xlib library to control keyboard input. For each key press, the FVP generates a physical interrupt. We run workloads that generate 1000 and 10000 interrupts.

GPU. The FVP supports a limited non-functional implementation of a Mali G76 GPU (No-Mali) [8]. The GPU cannot perform any computation but only exposes an MMIO layout and can send interrupts. So, we run the official Midgard Mali Kuft IRQ test suite [9]. The test suite triggers an MMIO write via the Mali driver and prompts the GPU to send 10000 interrupts.

UART. We attach the FVP UART PrimeCell PL011 peripheral to the VMs in our setup. This device facilitates serial communication which we control using an X11/Xlib library. We trigger workloads that generate 1000 and 10000 interrupts.

LED & Button. The FVP includes 8 LEDs and 8 User DIP switches (buttons) which can be controlled by software using system registers `SYS_LED` and `SYS_SW` respectively. To test their functionality, we write 1000 random values to these

registers and measure the total number of interrupts.

Mouse. On the FVP, the mouse works by capturing the mouse input in the FVP visualization component. Moving the mouse from the left to the right side of the screen raises 53 interrupts. A single mouse click triggers 2 interrupts. While we can verify that the mouse is functional, we cannot benchmark it as we are unable to perform this operation (for moving the mouse, or clicking) without user interaction.

B.2 Q1: Security Evaluation on FVP

We validate that DEVLORE enforces correct device isolation with 4 checks. These checks instantiate adversarial behaviors from Sec 7 and empirically validate that our implementation enforces DEVLORE security guarantees.

(a) Benign Interrupts. We run device workloads (GPU, UART, Mouse & Keyboard, LED & Button) that generate 50-10k interrupts (see Tab 2) and verify that all these device-raised interrupts reach the CVM correctly, as expected.

(b) Malicious Virtual Interrupts. We repeat the benign workloads but additionally modify the hypervisor inject 10k virtual interrupts that have no corresponding physical interrupt of a protected device. For this, we create a kernel module that invokes `kvm_vgic_inject_irq` on the KVM instance of the CVM (174 Loc). We report that DEVLORE rejects all injected malicious interrupts in the RMM and only delivers virtual interrupts to the CVM that are backed by a recorded physical interrupt, as expected.

(c) Malicious Physical Interrupts. We repeat the benign workloads but additionally modify the hypervisor to write to the GIC configuration to raise 10k physical interrupts for a protected device (Set-Pending register, `GICD_ISENDR` on the Distributor MMIO of the GIC, 168 LoC). We report that these malicious writes trap to the monitor and are blocked from reaching the GIC, as expected.

(d) Malicious Memory Accesses. We configure the FVP SMMU test engine [11] to issue memory accesses to the MMIO region of the protected device, including the registers for interrupt acknowledgement. The test engine is not attached to the CVM, so all accesses are unauthorized. We report that our implementation blocks these accesses with GPT violations. In addition, we modify the hypervisor to read from the MMIO region of the protected device directly, and report that these accesses are likewise rejected with GPT faults (176 Loc).

B.3 Case Study Setup on FVP

Below we detail our setup for each of the FVP case studies.

Terminal Based Browser. We cross-compile Lynx browser v2.9.2 [30] for Arm. We attach the UART to the realm VM and browse `en.wikipedia.org` in Lynx. On the landing page, we navigate to the *From Today’s featured Article* section and follow 4 subsequent links [70].

Table 4: Glmark2 with all scenes in their standard configuration for 20k frames.

Scene	Description	# Interrupts
buffer	Vertex buffer object throughput	2528
build	Shader compilation and setup	18678
bump	Per fragment lighting with bump mapping	20030
clear	Framebuffer clear performance	20035
conditionals	Shader branching and flow control	12906
desktop	Window compositing and blending effects	24793
effect2d	2D post processing filters	10085
function	Function call overhead in shaders	12902
ideas	Dynamic multi object scene rendering	14977
jellyfish	Animated mesh deformation workload	6164
loop	Shader loop execution stress test	12886
pulsar	Procedural animation and texture sampling	14212
refract	Environment mapped refraction rendering	25394
shading	Lighting and material computation cost	15124
shadow	Shadow mapping performance	16351
terrain	Height mapped terrain rendering	32335
texture	Texture sampling bandwidth	17514

IRC Client. We cross-compile a Console IRC Client [48] and connect to Miniircd Server v2.3 [56]. The server has 2 other users logged in. We exchange 6 messages with a user.

Text Editor. We cross-compile Nano text editor v8.1 [4] and add a new DNS entry to `/etc/hosts`.

B.4 Case Study Setup on Board

We cross-compile glmark2 [3] version 2023.01 for aarch64 Linux and run it in surfaceless mode on the Mali G610 GPU (OpenGL ES 3.1, Mesa 25.0.7-2). All 17 scenes execute unmodified in their standard configuration at 1920x1080 resolution for 20'000 frames and 30 runs in each environment: a realm-world CVM (**B**) and a realm-world CVM with DEVLORE interrupt protection (**I**). The GPU frequency is fixed to 800 MHz, and DVFS is disabled to avoid frequency-scaling effects. We use glmark2 in the default configuration with: `buf=32, r=8, g=8, b=8, a=8, depth=24, stencil=0, multisample anti-aliasing=0`. Tab 4 enumerates the benchmarks.